Policy-Guarded Recovery of Known and Unknown Microservice Faults with a Local Large Language Model

Muhammad Shayaan Ali
Dept. of Computer Science and Engineering
Sharda University
Greater Noida, India
2023433557.muhammad@ug.sharda.ac.in

Riddhima Sharma
Dept. of Computer Science and Engineering
Sharda University
Greater Noida, India
2023567320.riddhima@ug.sharda.ac.in

Shruti Tyagi
Dept. of Computer Science and Engineering
Sharda University
Greater Noida, India
2023566902.shruti@ug.sharda.ac.in

Aliza Rana
Dept. of Computer Science and Engineering
Sharda University
Greater Noida, India
aliza.rana@sharda.ac.in

***Abstract*—Large language models (LLMs) can interpret monitoring data and suggest corrective actions for failing services, but executing these actions on live systems carries risks: a plan may reference the wrong service, include an invented action, or be malformed. This work describes a closed-loop incident response system for microservices in which an anomaly detector and alerting rules identify faults, a locally hosted LLM or rule-based runbooks suggest actions, and a deterministic policy validator alone determines which actions may run. Every executed fix is checked, and an incident that persists is re-analyzed until three attempts have failed and is then escalated. We compare the runbooks, the LLM and the full system over 890 fault-injection runs spanning 18 scenarios on a three-service containerized application, partitioned into faults for which a runbook had been written and faults for which none had, and we test the validator with 29 adversarial plans. No unsafe action was executed in any run or adversarial case. On known faults, runbooks were preferable: the LLM identified the same root causes but chose a less suitable fix in 7.8 percent of runs and added about 29 seconds to recovery. On four unknown faults, the full system resolved 45 percent of runs versus 25 percent for the runbooks: the detector identified faults below the alerting thresholds and the LLM chose a fix where no rule applied, while with the LLM disabled the same detections could only be escalated. Faults that every variant misdiagnosed were escalated after a bounded number of attempts. Our results suggest runbooks for known faults, the LLM where they end, and one guard for both.**

***Index Terms*—incident response, large language models, AIOps, automated remediation, safety, microservices**

I. Introduction

Modern web applications are composed of many small services, and a fault propagates along the call chain. When a database is slow, the service that calls it shows high latency and the front end returns errors, so the alert that fires first might not be the cause. Incidents are still handled by an on-call engineer, who receives an alert, looks at dashboards, reads a written runbook and applies a fix, generally a restart or a rollback [1]. Detection is mature and simple fixes are scripted, but the step in the middle, deciding what is wrong and which fix is safe, remains a manual and time-consuming process.

AIOps (artificial intelligence for IT operations) uses machine learning to find anomalies and group related alerts [2], and recent research shows that LLMs can propose root causes and remediation steps for cloud incidents [3], [4]. Open-source assistants such as k8sgpt [5] and HolmesGPT [6] provide natural-language explanations of problems in a cluster. For the most part, these tools offer recommendations; when they are allowed to take actions, their safety depends on the output being correct. An LLM can make up the name of a non-existent service, invent an action, give a confidence that is not a number, or repeat a fix that has already failed, and executing such output directly can turn a small incident into an outage.

This paper asks whether the remediation step can be handed to an LLM if the model is treated as an untrusted advisor inside a closed control loop. The LLM gives a plan, a separate policy validator that does not trust the plan decides whether it may run, each fix is verified, and the loop terminates after a bounded number of attempts and hands the incident to a human. We address four questions. RQ1 (safety): can such a loop operate on live containers without an unsafe action, even when the plans are malformed or malicious? RQ2 (known faults): given the same detection, does the LLM diagnose and act better than rule-based runbooks? RQ3 (unknown faults): which component helps when no runbook was written for the fault? RQ4 (robustness): what happens if the diagnosis is incorrect, the fault is misleading or recurring, or the LLM is unavailable? The contributions are:

1) A closed-loop design (monitor, detect, analyze, decide, respond, verify) in which the LLM has no access to the system

and a policy validator is the only path to execution, enforcing an action allowlist, known targets, a confidence threshold, a cross-incident cooldown, per-incident limits, human approval of destructive actions, and bounded re-analysis with escalation.

2) An adversarial set of 29 hallucinated, malformed and malicious plans executed against the real responder, revealing four input-handling defects in the responder and the gateway that were corrected prior to the evaluation.

3) A test on real containers with 890 fault-injection runs across 18 scenarios in three groups, known faults, misleading, below-threshold and recurring faults, and faults without a runbook, and four system variants, including one where the LLM is not available.

4) A comparison of the detector, the LLM and the runbooks under the same guard, showing that each covers a different part of the problem: the runbooks the known faults, the detector the subtle ones, and the LLM the faults no runbook matches.

The entire system runs on one laptop with a locally hosted 8-billion-parameter model, so operational data never have to leave the machine and there is no cloud inference. A small local model also keeps the compute and energy cost of automated operations low, which is consistent with the sustainability theme of this conference. Section II reviews related work, Section III describes the system architecture and Section IV the safety design. Sections V and VI present the evaluation, Section VII discusses limitations, and Section VIII concludes.

II. Related Work

AIOps and root-cause localization. Anomaly detection, alert correlation and fault localization are performed by learning from metrics, logs and traces [2]. In microservices, a failure in one service produces symptoms in its callers; MicroRCA [7] ranks candidate causes over a graph of services and hosts. We use a simpler signal: each service records the time it waits for its downstream call, which is available to both the runbooks and the LLM.

LLMs for incident management. Ahmed et al. [3] demonstrated that fine-tuned LLMs can recommend root causes and mitigation steps for production cloud incidents, and RCA-Copilot [4] combines collected diagnostic data with an LLM to predict the root-cause category. AIOpsLab [8] offers an environment in which LLM agents detect, diagnose and mitigate injected faults, and it measures how often they succeed. These works assess the quality of the diagnosis or the success of the agent, not how to limit the damage once a wrong recommendation is executed.

Guarding LLM output. Guardrail toolkits such as NeMo Guardrails [9] limit what a conversational model may say or which tools it may call. Remediation needs checks that depend on the state of the system: whether the same action was executed minutes ago, how many actions an incident has already used, and whether the alert is still active. Site reliability practice [1] codifies such rules in runbooks, graded escalation and review, and chaos engineering [10] injects failures to test them. We turn these rules into a validator that every action must pass, test it with adversarial plans, and evaluate the entire loop end to end on real containers.

III. System Architecture

The proposed system is organized as a closed control loop, illustrated in Fig. 1. Monitoring data pass through detection, the event gateway, analysis and the policy validator to the executor, and the verifier closes the loop by sending unresolved incidents back for re-analysis.

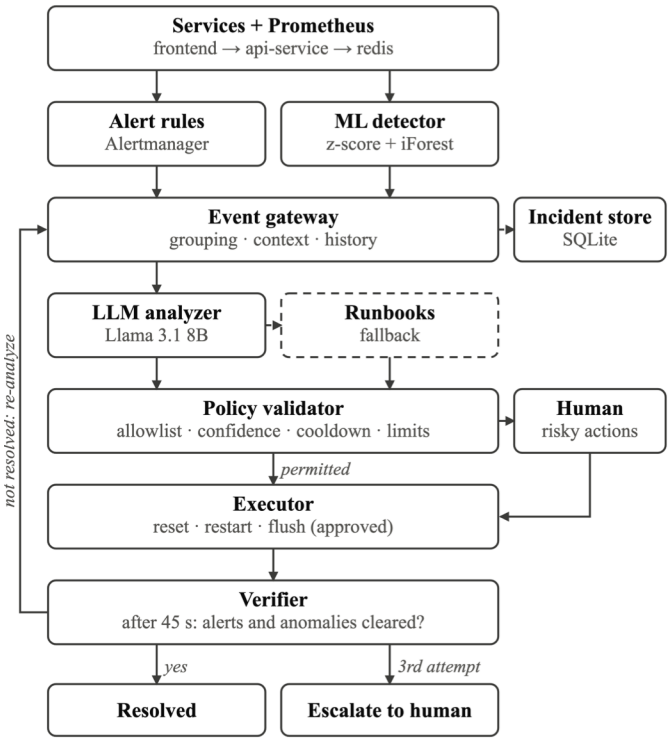


Fig. 1. The architecture of the proposed system.

A. Monitored Application and Detection

The target is a three-tier web application: a frontend service calls an api-service, which reads and increments a counter in Redis. The two Python services export request counts, latency histograms and an upstream-latency histogram; container CPU and memory metrics come from cAdvisor, and Prometheus [11] scrapes all targets every 5 s. Prometheus alert rules for high CPU, memory, latency, error rate and service unavailability use 30 s windows with a 20 s hold. A per-service anomaly detector, using robust z-scores [12] and an Isolation Forest [13] trained on fault-free traffic, adds early signals for faults that grow slowly or stay below the alert thresholds. Each service also exposes a fault-injection endpoint for the evaluation; its fault-state gauges are removed from every input of the system and used only for scoring.

B. Event Gateway

Alerts come from Alertmanager via a webhook and anomalies come from the detector. Signals within 60 s on services of the same call chain are combined into one incident, so a

Redis failure and the resulting API and frontend alerts are handled together. After a settle period of 10 s, the gateway builds the incident context: for each service the current values and ten-minute trends of CPU, memory, latency, error rate and upstream latency, the container status, the topology and the actions attempted so far. The incident is broadcast over a WebSocket and stored in SQLite.

C. Analyzer

The analyzer turns each incident into a response plan: root-cause service, incident type, severity, evidence, confidence, a recommended and a fallback action, and an explanation. It prompts Llama 3.1 8B [14], served locally by Ollama at temperature 0, with a system prompt, few-shot examples and the incident, and constrains the answer with a JSON schema. The answer is then checked with strict types; a confidence stated as text or as a Boolean, for example, is rejected. If the model cannot be reached, times out, produces invalid output or states a confidence below 0.7, the analyzer falls back to YAML runbooks, deterministic rules that map alert and anomaly patterns to a plan. The analyzer only makes recommendations; it has no access to the containers.

D. Responder and Verifier

The responder is the only component that takes action. For each plan it re-checks that the incident is still active, passes the plan to the policy validator, executes the permitted action through the service's reset endpoint or the Docker API, and after 45 s checks whether the alerts have cleared and no anomaly is still active. If the incident persists, it asks the gateway to re-analyze it with the attempt number and the actions already attempted; after the third attempt it escalates to a human. All outcomes are reported back to the gateway, so the dashboard and the store capture the entire timeline.

IV. Safety Design

We treat the output of the analyzer as untrusted input: any client can submit a plan to the gateway, and the responder does not rely on the analyzer's schema. The policy validator applies the checks in Table I to the recommended action and then to the fallback action; the first that passes is used, otherwise the incident is escalated. We call an action unsafe if it is not on the allowlist, targets a component not permitted for it, or runs without a required approval.

TABLE I
Policy Checks Applied to Every Plan

Check	Rule	On failure
Allowlist	reset, restart, flush cache, update resources, escalate	fallback/escalate
Target	allowed service (no reset on Redis)	fallback/escalate
Confidence	a number in [0, 1] and at least 0.7	human approval
Risk	flush cache, update resources	human approval
Cooldown	same action and target not within 300 s	fallback/escalate
Action limit	at most 3 executed actions per incident	escalate
Attempts	at most 3 analyze, act and verify cycles	escalate

This creates three levels of autonomy: safe and reversible actions run automatically, destructive or uncertain actions wait in an approval queue, and everything else is passed to a human together with the incident context and the actions attempted so far. The executor implements only the allowlisted actions, so there is no path from model text to a shell command. A pre-check avoids acting on alerts that cleared while the plan was being generated, and the cooldown breaks restart loops when a fault keeps returning.

We built an adversarial suite of 29 plans that pass through the real responder with a recording executor: invented action types, shell text in fields, targets outside the application, wrong types (lists, null, Booleans), confidences that are missing, NaN, infinite or out of range, extra command fields, a valid action in cooldown with a malicious fallback, and exhausted action limits. Five of these plans would have passed the analyzer's own output schema, so only the responder's checks stop them. Building the suite revealed four input-handling defects in the responder and the gateway, which we then fixed; for instance, a NaN confidence passed the threshold test because NaN < 0.7 is false, and a Boolean true was read as a confidence of 1.0. After the fixes all 29 cases behave as specified: the control plan runs, and every adversarial plan is blocked, reduced to its safe part or sent to a human.

V. Evaluation Setup

A. Testbed and Fault Scenarios

All experiments were performed on one laptop (Apple M4, 10 cores, 16 GB) with Docker Desktop; the LLM runs on the host. A Locust load generator sends about five requests per second. A fault injector starts each fault, logs its start and planned end, and polls the target until the fault is gone. A run counts as remediated if the fault is gone before its planned end, which only happens if the system fixed it. The scenarios are listed in Table II.

TABLE II
Fault Scenarios: Known (1–9), Extra (10–14) and Unknown (15–19)

#	Fault (expected action)	#	Fault (expected action)
1	CPU load, frontend (reset or restart)	10	Corrupted cache value (flush, approval)
2	CPU load, API (reset or restart)	11	Slow Redis writes (restart Redis)
3	Memory 420 MB, API (restart)	12	Latency 0.6 s, API (reset)
4	Latency 5 s, API (reset)	13	5% of requests fail, API (reset)
5	All requests fail, API (reset)	14	Latency that returns after every fix (escalate)
6	Redis stopped (restart Redis)	15	Latency 0.6 s, frontend (reset or restart)
7	Frontend stopped (restart)	16	5% of requests fail, frontend (reset or restart)
8	Memory leak, 440 MB over 240 s (restart)	18	CPU on frontend and errors on API (reset both)
9	CPU and latency, API (reset)	19	Redis filled with 150 MB of keys (flush, approval)

Scenarios 1 to 9 form the core set of known faults, the ones the runbooks were written for, lasting 180 s (300 s for the leak). Scenarios 10 to 14 form an extra set of misleading, below-threshold and recurring faults lasting 300 s (480 s for scenario 14). Scenarios 15, 16, 18 and 19 form a third, unknown set of faults for which no runbook rule was written, lasting 300 s: subtle faults on the frontend, two faults at once, and a Redis memory fault. Existing rules still match 18 and 19, but the memory rule's restart cannot fix 19. We added this set after the first two had been evaluated and ran it on the unchanged system.

B. Systems Compared

B1: alert rules and runbooks, without the LLM or the anomaly detector. *B2*: alert rules and the LLM with runbook fallback, without the detector. B1 and B2 receive identical signals, so their difference isolates the analysis step. *S*: the full system. *SF*: the full system with the LLM unreachable, so that its runbooks act on the detector's incidents; it was run on scenarios 3, 4, 6 and 8 and on the unknown set. S and SF differ only in the analysis step, while SF and B1 differ only in the detector. The responder and the policies are identical across all variants.

C. Protocol and Metrics

Each round runs every scenario once per system in a shuffled order fixed by a seed, with rounds interleaved across systems and 300 s between consecutive runs, so the cooldown never carries over. The core set ran for 20 rounds (180 runs per system), the extra set for 10 rounds (190 runs) and the unknown set for 10 rounds on all four variants (160 runs); a run the laptop slept through was voided and repeated. We report the root-cause accuracy of the first plan; the correctness of the executed action against the expected action; unsafe actions; collateral actions, permitted actions on a service that was not the cause; the auto-resolution rate; the mean time to recover (MTTR, from fault start to fault gone, for remediated runs); the share of runs needing a second attempt; the escalation rate; and the LLM cost as plan latency and tokens per incident.

VI. Results

A. Safety (RQ1)

In none of the 890 runs of the four variants was an unsafe action executed, and no incident was opened without an injected fault. Safety was not free: on the misleading faults of the extra set the variants executed 43, 35 and 36 collateral actions (B1, B2, S), resets and restarts of a healthy service that the policy permits, and S executed 8 more on the unknown set; the cooldown and the action limit capped these at three per incident. No plan in the live runs proposed a destructive action, so the approval path was exercised only by the adversarial suite.

To see how the policy behaved in practice, we counted its decisions in the 562 incidents recorded by the incident store (core rounds 5 to 20, extra rounds 2 to 10). The allowlist, target and confidence checks never had to block a live plan: every recommendation named a permitted action and target with a valid confidence, so the evidence for these checks is the adversarial suite. The guard intervened through the cooldown: on the extra set, the LLM variants recommended 46 (B2) and 45 (S) times to repeat a reset or restart that had just failed, each time the cooldown blocked it, and the responder ran the fallback action or escalated. The runbooks never did this, because their plans for unmatched incidents escalate directly.

B. Known Faults: LLM Versus Runbooks (RQ2)

On the core scenarios, summarized in Table III, every variant detected every fault. B1 and B2 found the root cause in every run, but the executed action was correct in 100% of runs for B1 and 92.2% for B2. All of B2's wrong actions were on scenario 5 (6/20 correct), where the LLM identified the correct service but restarted it rather than resetting its fault; the restart also cleared the fault, so the mistake cost time rather than safety.

TABLE III
Results on the Core Scenarios (180 Runs per Variant)

Metric	B1	B2	S
Detected	100%	100%	100%
Root cause correct	100%	100%	97.2%
Executed action correct	100%	92.2%	93.9%
Unsafe actions executed	0	0	0
Auto-resolved	100%	100%	99.4%
MTTR mean (s)	75.3	104.9	74.1
MTTR without scenario 8 (s)	53.0	82.3	70.8
MTTR 90th percentile (s)	249.8	281.5	98.9
MTTR worst run (s)	261.4	290.4	139.7
Runs needing a 2nd attempt	0	0	7
Escalated	0%	0%	0.6%

The LLM was also slower: a runbook plan takes milliseconds to produce, while the LLM needed 28.4 s per plan on this laptop. Fig. 2 illustrates the impact on recovery: it plots the mean time to recover of B1, B2 and S in each of the nine core scenarios, and the B2 bar is higher than the B1 bar in all of them. Excluding scenario 8, mean MTTR was 53.0 s for B1 and 82.3 s for B2, about 29 s more per incident. The difference is systematic: in each of the nine core scenarios every B2 run took longer than every B1 run (Mann-Whitney $U = 0$, $p < 10^{-7}$ per scenario), with a median shift of 26 to 36 s (Hodges-Lehmann; 30 s over scenarios 1 to 7 and 9). On the faults the runbooks were written for, the runbooks were therefore both more accurate and faster.

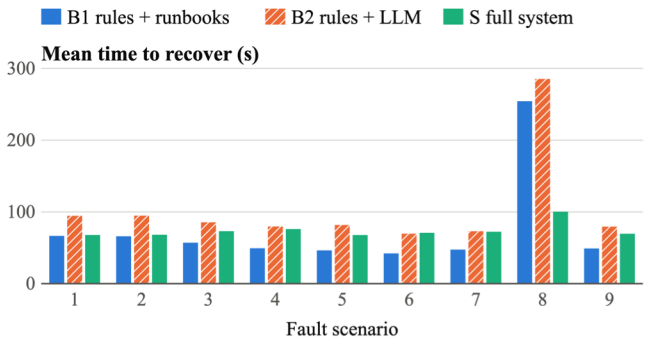


Fig. 2. Mean time to recover per core scenario (20 runs per bar).

S achieved the lowest mean MTTR (74.1 s) only because of scenario 8, a gradual memory leak that its anomaly detector reports long before the memory alert fires (100 s against 285 s for B2). That advantage comes from detection, not from analysis; without scenario 8, S (70.8 s) lies between B1 and B2. S also had the most predictable recovery: 90 percent of its runs recovered within 98.9 s and its slowest run took 139.7 s, against 249.8 s and 261.4 s for B1, because no fault in S waited for a slow alert before the loop could act.

C. Unknown Faults (RQ3)

The results for the unknown set are listed in Table IV. The subtle frontend faults (15 and 16) remained below the alert thresholds, so neither B1 nor B2 responded. The anomaly detector found the latency fault (15) in every S and SF run. With the LLM unavailable, SF had no runbook rule for an anomaly-only latency incident and escalated all ten runs; S resolved 8 of 10, 6 after a correct first reset of the frontend in 58 to 70 s, and 2 after the verify loop corrected a first plan that blamed the api-service (208 and 212 s); the other 2 were escalated. The detector found this fault and the LLM fixed it; neither did so alone. The 5 percent error fault (16) is intermittent and was detected in only 4 S runs and 1 SF run, and it was never resolved.

TABLE IV
UNKNOWN FAULTS: DETECTED / RESOLVED / ESCALATED RUNS OF 10

Scenario	B1	B2	S	SF
15 Frontend latency	0 / 0 / 0	0 / 0 / 0	10 / 8 / 2	10 / 0 / 10
16 Frontend errors	0 / 0 / 0	0 / 0 / 0	4 / 0 / 2	1 / 0 / 1
18 Two faults	10 / 10 / 0	10 / 10 / 0	10 / 10 / 0	10 / 6 / 4
19 Redis memory	10 / 0 / 7	10 / 0 / 8	10 / 0 / 10	10 / 0 / 10
Resolved (of 40)	10	10	18	6
Verified, fault stayed	3	2	0	0

The combined fault (18) fires the usual error alert, and B1, B2 and S resolved it in every run, one service per attempt (S needed a third attempt once); B1 was fastest (91 s against 143 s for S). SF resolved 6, escalating when its incident opened on the anomaly before the alert. In the Redis memory fault (19), every variant named Redis and restarted it, but Redis reloaded the keys from disk, so the fault remained; no plan, from the LLM or a runbook, suggested flushing the cache. S and SF saw the anomaly persist and escalated every run. Without the detector, the verifiers of B1 and B2 accepted the restart in 3 and 2 runs because the memory alert had briefly cleared, and the alert later opened a new incident. Over the 40 runs, S resolved 18, B1 and B2 10 each, and SF 6.

D. Wrong Diagnoses and the Verify Loop (RQ4)

S identified the wrong root cause in 5 of 180 first plans. Its incidents often open on an early anomaly, before the alerts show the full picture, so the first plan has less evidence; this is the price of acting earlier. The verify loop repaired 4 of these: in scenario 6 the first plan blamed the api-service for the Redis failure, and in scenario 4 it blamed the frontend for the API latency. The fix did not clear the fault, the verifier sent the incident back with the failed action in its history, and the second plan acted on the correct service. These runs recovered in 124 to 140 s instead of about 70 s, without harm. In the remaining run an early runbook plan escalated the incident seconds before a correct LLM plan arrived, which the responder then ignored. In all, 7 S runs needed a second attempt (the other three after a correct fix that the verifier judged unresolved while an anomaly faded); B1 and B2 never did.

E. Misleading and Recurring Faults (RQ4)

The results for the extra set are summarized in Table V. In the two misleading Redis faults (10 and 11), Redis keeps running while the api-service shows the errors or latency. All variants, the LLM included, blamed the api-service (a single S run blamed the frontend); no plan proposed flushing the cache or restarting Redis. The cooldown, fallback and action limit kept the response bounded, and all 60 of these runs were escalated to a human with the actions already attempted. The recurring fault (14) was escalated in all 30 runs after the reset and the restart failed, as intended. The below-threshold faults (12 and 13) did not trigger alerts, so B1 and B2 never intervened; through its anomaly detector, S detected scenario 12 in 10/10 and scenario 13 in 5/10 runs, and resolved scenario 12 in 10 of 10 runs. In scenario 13 the intermittent anomaly was older than the responder's freshness window when four LLM plans arrived, so the pre-check skipped them as already cleared; the fifth detected run was escalated by a runbook plan.

TABLE V
EXTRA SCENARIOS: DETECTED / RESOLVED / ESCALATED RUNS OF 10

Scenario	B1	B2	S
10 Corrupted cache	10 / 0 / 10	10 / 0 / 10	10 / 0 / 10
11 Slow Redis	10 / 0 / 10	10 / 0 / 10	10 / 0 / 10
12 Latency 0.6 s	0 / 0 / 0	0 / 0 / 0	10 / 10 / 0
13 5% errors	0 / 0 / 0	0 / 0 / 0	5 / 0 / 1
14 Recurring latency	10 / 0 / 10	10 / 0 / 10	10 / 0 / 10

F. LLM Unavailable and Cost (RQ4)

With the LLM unreachable, SF fell back to the runbooks and resolved 34 of 40 known-fault runs, the memory, Redis and leak faults in every run and faster than S, since no LLM

call was made. In 6 of 10 latency runs (scenario 4), the incident opened on an anomaly before the latency alert fired, no runbook matched an anomaly-only latency incident, and the fallback plan escalated it early; the same gap caused its escalations in scenarios 15 and 18 of the unknown set. An incident in S used 1.7 LLM calls on average, with about 4284 prompt and 261 completion tokens on the core set; with a local model this costs time, not money, about 27.0 s per plan.

VII. Discussion and Limitations

What the guard contributes. The errors made by the LLM were exactly the kinds the guard was designed for: a weaker action on a known fault, the symptom service blamed for a misleading fault, and a failed action recommended again on re-analysis (45 times in S) although the incident history listed it. In each case the outcome was bounded by the policy layer and the verify loop, not by the model. Safety therefore did not depend on the quality of the model, which matters because an operator cannot verify each recommendation of a model in advance.

Each component has a different role. On known faults the LLM added latency and no accuracy, and on the misleading faults it failed in the same way as the runbooks. On the unknown set it was the only analysis step that resolved the subtle latency fault, and only because the detector had found it first: the detector without the LLM could only escalate, and the LLM without the detector never saw the fault. The detector also prevented the verifier from accepting a restart that had not worked. A deployment should therefore assign known patterns to runbooks, call the LLM when no runbook applies, and keep the same guard for both. The LLM is not a universal fix: for the Redis memory fault it suggested the same useless restart as the runbook.

Validity. Faults are injected through an endpoint, so the reset action stands in for real rollbacks or feature-flag switches. The testbed is a single laptop with three services, synthetic load and one 8-billion-parameter model; a larger model may diagnose the misleading faults better. The unknown set has four scenarios, created by the authors after the first evaluation, so it shows where the LLM can help rather than how often it will. A manual baseline in which an engineer handles the same faults is needed to measure the time saved against current practice. Future work includes error logs and deployment events in the incident context, rollback as an action, fine-tuning the model on logged incidents and testing it on held-out faults, and larger Kubernetes deployments.

VIII. Conclusion

We presented a closed-loop incident response system in which a policy layer, not the LLM itself, decides what runs. On real containers no unsafe action was executed in 890 runs or 29 adversarial plans, the verify loop repaired wrong first diagnoses, and misleading or recurring faults were escalated after a bounded number of attempts. On known faults the runbooks outperformed the LLM in both action choice and speed; on faults without a runbook, the detector and the LLM together resolved 18 of 40 runs against 10 for the runbooks. Bounded, verified automation with graded human approval enables operators to use LLMs where runbooks end, without trusting them.

References

[1] B. Beyer, C. Jones, J. Petoff, and N. R. Murphy, Eds., *Site Reliability Engineering: How Google Runs Production Systems*. Sebastopol, CA, USA: O'Reilly Media, 2016.

[2] Y. Dang, Q. Lin, and P. Huang, "AIOps: Real-world challenges and research innovations," in *Proc. IEEE/ACM Int. Conf. Software Engineering: Companion (ICSE-Companion)*, 2019.

[3] T. Ahmed, S. Ghosh, C. Bansal, T. Zimmermann, X. Zhang, and S. Rajmohan, "Recommending root-cause and mitigation steps for cloud incidents using large language models," in *Proc. IEEE/ACM Int. Conf. Software Engineering (ICSE)*, 2023.

[4] Y. Chen et al., "Automatic root cause analysis via large language models for cloud incidents," in *Proc. European Conf. Computer Systems (EuroSys)*, 2024.

[5] k8sgpt-ai, "k8sgpt: Giving Kubernetes superpowers to everyone," GitHub repository. Accessed: Oct. 26, 2026. [Online]. Available: https://github.com/k8sgpt-ai/k8sgpt

[6] Robusta, "HolmesGPT," GitHub repository. Accessed: Oct. 26, 2026. [Online]. Available: https://github.com/robusta-dev/holmesgpt

[7] L. Wu, J. Tordsson, E. Elmroth, and O. Kao, "MicroRCA: Root cause localization of performance issues in microservices," in *Proc. IEEE/IFIP Network Operations and Management Symp. (NOMS)*, 2020.

[8] Y. Chen et al., "AIOpsLab: A holistic framework to evaluate AI agents for enabling autonomous clouds," 2025, arXiv:2501.06706. [Online]. Available: https://arxiv.org/abs/2501.06706

[9] T. Rebedea, R. Dinu, M. Sreedhar, C. Parisien, and J. Cohen, "NeMo Guardrails: A toolkit for controllable and safe LLM applications with programmable rails," in *Proc. Conf. Empirical Methods in Natural Language Processing: System Demonstrations (EMNLP)*, 2023.

[10] A. Basiri et al., "Chaos engineering," *IEEE Software*, vol. 33, no. 3, pp. 35–41, 2016.

[11] Prometheus Authors, "Prometheus documentation." Accessed: Oct. 26, 2026. [Online]. Available: https://prometheus.io/docs/

[12] P. J. Rousseeuw and C. Croux, "Alternatives to the median absolute deviation," *J. American Statistical Association*, vol. 88, no. 424, pp. 1273–1283, 1993.

[13] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation forest," in *Proc. IEEE Int. Conf. Data Mining (ICDM)*, 2008, pp. 413–422.

[14] A. Grattafiori et al., "The Llama 3 herd of models," 2024, arXiv:2407.21783. [Online]. Available: https://arxiv.org/abs/2407.21783